ENTERPRISE DIGITAL BANKING SYSTEM

05

Security Documentation

Authentication flow, authorization matrix, sensitive data, threat model, secrets management and audit policy.

Document	05 — Security Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Security Architecture
2. 2. Authentication Flow
3. 3. Authorization Matrix
4. 4. Sensitive Data
5. 5. Threat Model
6. 6. Secrets Management
7. 7. Audit Policy
8. 8. Security Review Checklist

1. Security Architecture



Figure 1 — Defence in depth: each layer catches what the previous one cannot

Control	Implementation	Status
Password storage	`BCryptPasswordEncoder` (salted, cost 10) in `SecurityBeansConfig`	BUILT
Authentication	Stateless JWT (jjwt 0.12.6, HMAC-SHA256), subject = e-mail	BUILT
Session	`SessionCreationPolicy.STATELESS` ; CSRF disabled (no cookies used)	BUILT
401 handling	`JwtAuthenticationEntryPoint` returns JSON `ErrorResponse`	BUILT
Input validation	Bean Validation on every request DTO (`@NotBlank` , `@Pattern` , `@DecimalMin` ...)	BUILT
SQL injection	Spring Data JPA derived queries only — parameterised by construction	BUILT
Error leakage	Catch-all handler returns generic 500 message, no stack traces	BUILT
Audit trail	`AuditService` on auth, account, transaction, beneficiary events	BUILT
Concurrency	`@Version` optimistic locking on accounts	BUILT
Secrets	Env vars `DB_PASSWORD` , `JWT_SECRET` , `MAIL_PASSWORD`	BUILT
Role-based authorization	`@EnableMethodSecurity` + `@PreAuthorize` / URL rules	PLANNED
Resource ownership	Customer may only access own accounts, loans, beneficiaries	PLANNED
Rate limiting	Nginx + Bucket4j/Redis (doc 09)	PLANNED
TLS	Nginx / AWS ALB with ACM certificate	PLANNED
Refresh tokens, password reset	DTOs exist; endpoints pending	PLANNED
CORS	`CorsConfigurationSource` bean with explicit origins (only if a browser frontend is added)	PLANNED

2. Authentication Flow

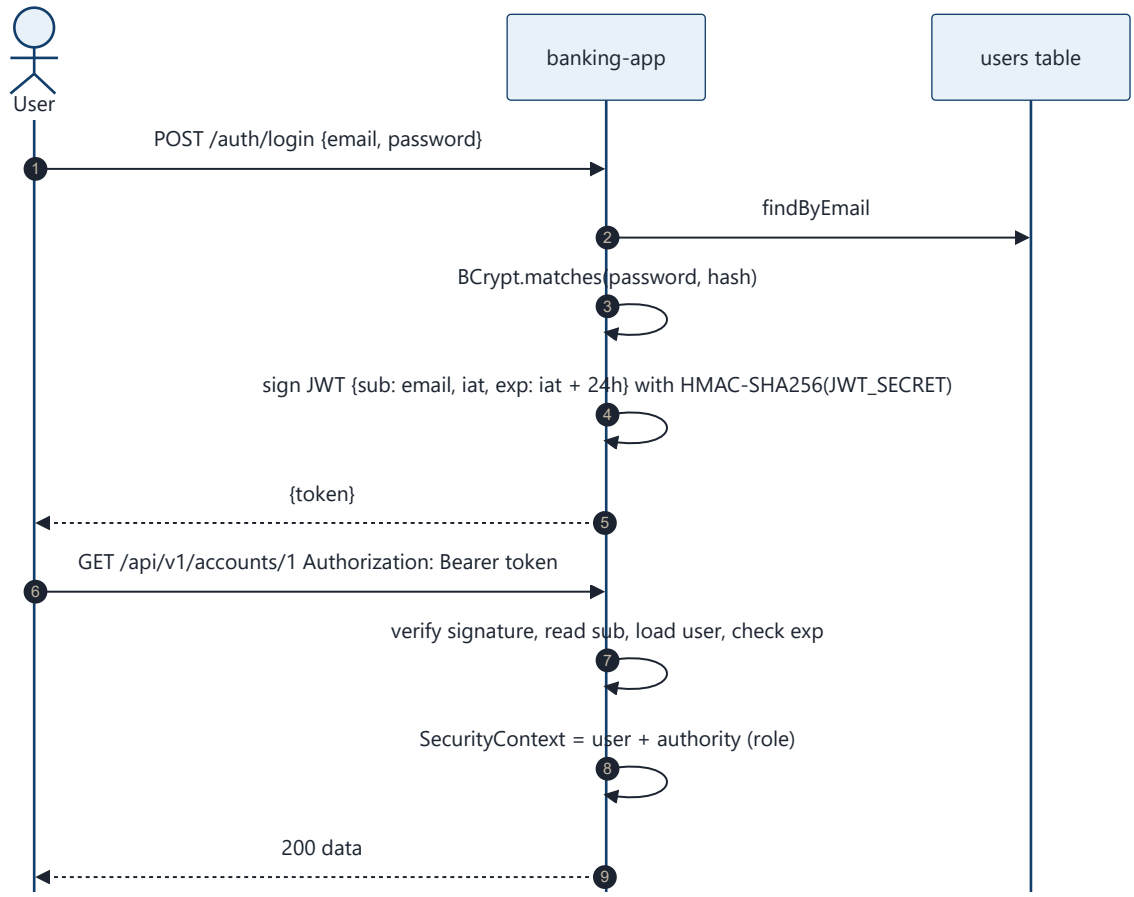


Figure 2 — Login and authenticated call

2.1 Token format

```
Header : { "alg": "HS256" }
Payload: { "sub": "hemant@example.com", "iat": 1760366400, "exp": 1760452800 }
Signature: HMACSHA256(base64url(header) + "." + base64url(payload), JWT_SECRET)
```

- `JWT_SECRET` must be at least 256 bits (32+ random bytes) — `Keys.hmacShaKeyFor` rejects shorter keys.
- Expiry: `jwt.expiration` ms (default 86 400 000 = 24 h). Target: 15 min access token + 7 day refresh token **PLANNED**.
- The token carries no role claim; the role is re-read from the database on every request, so a role change takes effect immediately.
- Tokens cannot be revoked before expiry **GAP**. Planned: refresh-token table + deny-list of `jti` in Redis on logout.
- `UserStatus` (BLOCKED / SUSPENDED) is not checked at login or in `CustomUserDetails.isEnabled()` **GAP**.

2.2 Password policy

Rule	Current	Target
Minimum length	8	10

Complexity	none	upper, lower, digit, symbol
Hashing	BCrypt cost 10	BCrypt cost 12
Lockout	none	5 failed logins → 15 min lock (via rate limiter + LOGIN_FAILED audit)
Login error messages	distinguish unknown e-mail (404) from wrong password (401) — allows user enumeration **GAP**	single generic 401 “Invalid email or password”

3. Authorization Matrix

Target role permissions. **Today** every authenticated user may call every endpoint; this matrix is to be enforced with `@PreAuthorize` **PLANNED**. "Own" = only resources belonging to the caller's customer profile.

Endpoint group	CUSTOMER	EMPLOYEE	MANAGER	ADMIN	AUDITOR
POST /auth/register, /auth/login	public	public	public	public	public
Customers: create / update	own	yes	yes	yes	—
Customers: list all	—	yes	yes	yes	read
Customers: complete KYC	—	yes	yes	yes	—
Customers: delete	—	—	—	yes	—
Accounts: view	own	yes	yes	yes	read
Accounts: open / update	—	yes	yes	yes	—
Accounts: freeze / close	—	—	yes	yes	—
Transactions: deposit	—	yes	yes	yes	—
Transactions: withdraw / transfer	own source	yes	yes	yes	—
Transactions / ledger: view	own	yes	yes	yes	read
Beneficiaries: add / rename / delete	own	—	—	yes	—
Beneficiaries: activate	own (after OTP, planned)	yes	yes	yes	—
Loans: apply	own	yes	yes	yes	—
Loans: approve / reject	—	—	yes	yes	—
EMIs: view / pay	own	yes	yes	yes	read
Audit logs	—	—	—	yes	yes
Actuator / metrics	internal network only (Nginx `deny all`)				

```
// SecurityConfig (target)
@EnableMethodSecurity
...
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/audit/**").hasAnyAuthority("ADMIN", "AUDITOR")
    .requestMatchers(HttpMethod.PATCH, "/api/v1/loans/*/approve", "/api/v1/loans/*/reject")
        .hasAnyAuthority("MANAGER", "ADMIN")
    .anyRequest().authenticated())

// Ownership check example
@PreAuthorize("hasAnyAuthority('EMPLOYEE','MANAGER','ADMIN') or @ownership.ownsAccount(#accountId)")
public AccountResponse getAccountById(Long accountId) { ... }
```

> Authorities are the plain role name (`new SimpleGrantedAuthority(user.getRole().name())`), so rules must use `hasAuthority("ADMIN")` , not `hasRole("ADMIN")` (which expects the `ROLE_` prefix).

4. Sensitive Data

Data	Location	Classification	Protection
Password	users.password	Secret	BCrypt hash only; never returned, never logged, never audited
JWT	Client storage, Authorization header	Secret	HTTPS only; short expiry; never logged
Aadhaar number	customers.aadhaar_number	Highly sensitive PII	Not returned in any response DTO. Target: column encryption (JPA `AttributeConverter` with AES-GCM) + masked display `XXXX-XXXX-1234`
PAN	customers.pan_number	Sensitive PII	Not returned in responses; target: encryption at rest
Name, DOB, phone, address, e-mail	users, customers	PII	Access restricted by authorization matrix; e-mail appears in Kafka events (internal network only)
Balances, transactions	accounts, transactions, ledger_entries	Confidential financial	Ownership checks (planned); append-only history
Client IP	audit_logs.ip_address	Personal data	Audit only
Card numbers	cards (planned)	PCI DSS	Never store full PAN/CVV; store token + last 4 digits only

5. Threat Model

STRIDE analysis of the main assets (money, credentials, PII) across the trust boundaries: Internet → edge, edge → app, app → data stores, app → Kafka → notification-service.

#	STRIDE	Threat	Mitigation	Status
T1	Spoofing	Password brute force / credential stuffing on `/auth/login`	Nginx 5 req/min per IP + app filter; LOGIN_FAILED audit; lockout	PLANNED
T2	Spoofing	Forged JWT	HMAC-SHA256 signature with ≥ 256-bit secret from env	BUILT
T3	Spoofing	Stolen token reused	HTTPS; short expiry + refresh tokens; revocation list	PLANNED
T4	Tampering	Amount / account id manipulated in request	Server-side validation, balance checks, account status checks	BUILT
T5	Tampering	Lost update / double spend through concurrent requests	`@Version` optimistic locking; single DB transaction per operation	BUILT
T6	Tampering	SQL injection	JPA parameter binding; no native string SQL	BUILT
T7	Repudiation	User denies making a transfer	Audit log with user, IP, timestamp; immutable ledger	BUILT
T8	Repudiation	Spoofed `X-Forwarded-For` hides real IP in audit	Nginx overwrites header; `server.forward-headers-strategy=native`; read `getRemoteAddr()` only	PLANNED
T9	Info disclosure	**IDOR**: customer reads another customer's account by changing the id	Ownership checks (section 3)	GAP
T10	Info disclosure	Stack traces / SQL in error responses	Generic 500 handler	BUILT
T11	Info disclosure	Secrets committed to git	Env vars; `.env` in `.gitignore`; secret scanning	BUILT
T12	Info disclosure	User enumeration through different login errors	Single generic message	GAP
T13	Info disclosure	Actuator endpoints exposed publicly	Nginx `deny all` on `/actuator/`	PLANNED
T14	Denial of service	Request floods / huge bodies	Nginx rate limits, `client_max_body_size 1m`, timeouts	PLANNED
T15	Denial of service	Unbounded list endpoints (history, audit)	Pagination	PLANNED
T16	Elevation of privilege	Customer approves own loan / reads audit log	Role-based authorization	GAP
T17	Elevation of privilege	Self-registration as ADMIN	Register always assigns CUSTOMER; role not accepted from client	BUILT
T18	Tampering	Forged Kafka events / duplicate delivery	Kafka on private network; idempotency key on notification; SASL/TLS on brokers in prod	BUILT / PLANNED

6. Secrets Management

Secret	Variable	Used by	Default
Database password	`DB_PASSWORD`	both services	none (required)
Database user	`DB_USERNAME`	both services	`root`
JWT signing key	`JWT_SECRET`	banking-app	none (required, ≥ 32 bytes)
JWT lifetime	`JWT_EXPIRATION`	banking-app	86400000
SMTP credentials	`MAIL_USERNAME` , `MAIL_PASSWORD`	notification-service	empty (e-mail delivery deferred)
Kafka address	`KAFKA_BOOTSTRAP_SERVERS`	banking-app	`localhost:9092`

6.1 Rules

- Never put real values in `application*.properties` , `docker-compose.yml` or the README; reference `${VAR}` only.
- Local: IntelliJ run-configuration env vars or a git-ignored `.env` file read by Docker Compose.
- AWS: Elastic Beanstalk environment properties, or AWS Secrets Manager / SSM Parameter Store for rotation.
- CI: GitHub Actions repository secrets.
- Generate the JWT secret with `openssl rand -base64 48` . Rotating it invalidates all issued tokens (users log in again).
- Verify nothing leaked: `git log --all -- .env` must be empty. Add `.env` to `.gitignore` PLANNED .
- Gmail SMTP requires an App Password, never the account password.

7. Audit Policy

7.1 What is audited

Event	Action	Entity	Old / new value	Status
User registered	CREATE	User	– / email	BUILT
Login success	LOGIN	User	–	BUILT
Login failed (unknown user / bad password)	LOGIN_FAILED	User	–	BUILT
Account created / updated / frozen / closed	CREATE / UPDATE	Account	status before / after	BUILT
Deposit / withdrawal / transfer	CREATE	Transaction	– / amount, accounts	BUILT
Beneficiary added / renamed / activated / removed	CREATE / UPDATE / DELETE	Beneficiary	nickname or status	BUILT
Customer created / updated / KYC / deleted	CREATE / UPDATE / DELETE	Customer	KYC status	PLANNED
Loan applied / approved / rejected; EMI paid	CREATE / UPDATE	Loan / Emi	status	PLANNED
Logout, password change / reset	LOGOUT / UPDATE	User	–	PLANNED

7.2 Rules

- Written in the same DB transaction as the change: if the business operation rolls back, so does its audit row (no phantom audit entries).
- **Never** store passwords, tokens, full Aadhaar/PAN or card numbers in `old_value` / `new_value`.
- Audit rows are append-only: no update/delete endpoint exists, and the DB user for the app should eventually have no `UPDATE/DELETE` grant on `audit_logs`.
- Read access: ADMIN and AUDITOR only (planned). Retention ≥ 8 years.
- Failed-login audits survive the exception that follows them because `UserServiceImpl` is not transactional — each `AuditService.log` call commits on its own. Keep it that way (or use `REQUIRES_NEW`) if `@Transactional` is ever added to the login method.

8. Security Review Checklist

Item	Result
CORS	Not configured; only needed for a browser frontend on another origin
JWT expiry	24 h — acceptable for a demo; shorten with refresh tokens in production
Password policy	Length only — simplified, documented
Secrets in config	Pass — all via env vars
Audit log leakage	Pass — no passwords logged
Error messages	Pass — generic 500
SQL injection	Pass — JPA only
Rate limiting	Gap — planned (doc 09)
Authorization / IDOR	Gap — highest priority fix before any real deployment
Dependency vulnerabilities	Planned: OWASP `dependency-check-maven` or GitHub Dependabot in CI